

WEEK-7 SKILL

1 . Process Synchronization and Child Process Monitoring Using waitpid()

Aim : To create multiple child processes and use the **waitpid()** system call to synchronize and monitor child processes, obtain their termination status, and ensure that the parent process correctly waits for its children.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t child1, child2;
    int status;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    // Create first child
    child1 = fork();

    if (child1 < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (child1 == 0)
    {
        printf("\nChild 1 started.\n");
        printf("Child 1 PID: %d\n", getpid());

        sleep(3);

        printf("Child 1 completed.\n");
        exit(10);
    }

    // Create second child
    child2 = fork();

    if (child2 < 0)
    {
```

```
if (child2 == 0)
{
    printf("\nChild 2 started.\n");
    printf("Child 2 PID: %d\n", getpid());

    sleep(5);

    printf("Child 2 completed.\n");
    exit(20);
}

// Parent waits for Child 1
printf("\nParent is waiting for Child 1...\n");

if (waitpid(child1, &status, 0) == -1)
{
    perror("waitpid failed");
    return 1;
}

if (WIFEXITED(status))
{
    printf("Child 1 terminated normally.\n");
    printf("Child 1 exit status: %d\n", WEXITSTATUS(status));
}

// Parent waits for Child 2
printf("\nParent is waiting for Child 2...\n");

if (waitpid(child2, &status, 0) == -1)
{
    perror("waitpid failed");
    return 1;
}

if (WIFEXITED(status))
{
    printf("Child 2 terminated normally.\n");
    printf("Child 2 exit status: %d\n", WEXITSTATUS(status));
}

printf("\nAll child processes have completed.\n");
printf("Parent process completed.\n");

return 0;
}
```

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ mkdir waitpid_practical
jhanasri@LAPTOP-1RN1M8E3:~$ cd waitpid_practical
jhanasri@LAPTOP-1RN1M8E3:~/waitpid_practical$ pwd
/home/jhanasri/waitpid_practical
jhanasri@LAPTOP-1RN1M8E3:~/waitpid_practical$ nano waitpid_demo.c
jhanasri@LAPTOP-1RN1M8E3:~/waitpid_practical$ gcc waitpid_demo.c -o waitpid_demo
jhanasri@LAPTOP-1RN1M8E3:~/waitpid_practical$ ls
waitpid_demo  waitpid_demo.c
jhanasri@LAPTOP-1RN1M8E3:~/waitpid_practical$ ./waitpid_demo
Parent process started.
Parent PID: 625

Child 1 started.
Child 1 PID: 626

Parent is waiting for Child 1...

Child 2 started.
Child 2 PID: 627
Child 1 completed.
Child 1 terminated normally.
Child 1 exit status: 10

Parent is waiting for Child 2...
Child 2 completed.
Child 2 terminated normally.
Child 2 exit status: 20

All child processes have completed.
Parent process completed.
jhanasri@LAPTOP-1RN1M8E3:~/waitpid_practical$
```

Result:

The `waitpid()` program was successfully implemented and executed in Ubuntu. Multiple child processes were created and their termination statuses were obtained successfully. The parent process correctly waited for each child process before completing.

Observation:

`waitpid()` successfully synchronized the parent with its child processes. Both child processes terminated normally with their respective exit statuses. The parent process monitored and collected the child statuses correctly, demonstrating proper process synchronization.

2 . To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

Aim : To understand how Linux uses the **PATH** environment variable to search for executable commands, verify executable permissions, handle missing commands, and test command resolution.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

int main()
{
    char *path;
    char *path_copy;
    char *dir;
    char full_path[1024];
    struct stat file_info;

    printf("==== PATH VARIABLE AND COMMAND RESOLUTION =====\n\n");

    /* Retrieve PATH variable */
    path = getenv("PATH");

    if (path == NULL)
    {
        printf("PATH variable is not set.\n");
        return 1;
    }

    printf("PATH variable:\n%s\n\n", path);

    /* Make a copy because strtok modifies the string */
    path_copy = strdup(path);

    if (path_copy == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Search directories:\n");

    dir = strtok(path_copy, ":");
```

```
while (dir != NULL)
{
    printf("%s\n", dir);
    dir = strtok(NULL, ":");
}

free(path_copy);

/* Test command resolution */
printf("\n==== COMMAND RESOLUTION =====\n");

const char *command = "ls";

path_copy = strdup(path);
dir = strtok(path_copy, ":");

while (dir != NULL)
{
    snprintf(full_path, sizeof(full_path), "%s/%s", dir, command);
    if (stat(full_path, &file_info) == 0)
    {
        printf("Command '%s' found at: %s\n", command, full_path);

        if (file_info.st_mode & S_IXUSR)
        {
            printf("Executable permission: YES\n");
        }
        else
        {
            printf("Executable permission: NO\n");
        }

        break;
    }

    dir = strtok(NULL, ":");
}

if (dir == NULL)
{
    printf("Command '%s' not found in PATH.\n", command);
}

free(path_copy);

/* Test missing command */
command = "abcxyz";

printf("\nTesting missing command: %s\n", command);
```

```
    dir = strtok(NULL, ":");
}

if (dir == NULL)
{
    printf("Command '%s' not found in PATH.\n", command);
}

free(path_copy);

/* Test missing command */
command = "abcxyz";

printf("\nTesting missing command: %s\n", command);

path_copy = strdup(path);
dir = strtok(path_copy, ":");

int found = 0;

while (dir != NULL)
{
    snprintf(full_path, sizeof(full_path), "%s/%s", dir, command);

    if (access(full_path, X_OK) == 0)
    {
        printf("Command found at: %s\n", full_path);
        found = 1;
        break;
    }

    dir = strtok(NULL, ":");
}

if (!found)
{
    printf("Command '%s' not found in PATH.\n", command);
}

free(path_copy);

printf("\n===== TEST COMPLETED =====\n");

return 0;
}
```

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ mkdir path_practical
jhanasri@LAPTOP-1RN1M8E3:~$ cd path_practical
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ pwd
/home/jhanasri/path_practical
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ nano path_resolution.c
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ gcc path_resolution.c -o path_resolution
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ ls
path_resolution  path_resolution.c
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ ./path_resolution
===== PATH VARIABLE AND COMMAND RESOLUTION =====
PATH variable:
/home/jhanasri/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Windows/System
32:/mnt/c/Windows:/mnt/c/Windows/System32/Wbem:/mnt/c/Windows/System32/Windows PowerShell/v1.0:/mnt/c/Program Files/Git/cmd:/mnt/c/Program Files/Nodejs:/mnt
c/Program Files (x86)/Common Files/Oracle/Java/javase/bin:/mnt/c/Program Files/Java/jdk-21/bin:/mnt/c/Program Files/Microsoft VS Code/bin:/mnt/c/Users/jhanasri
/AppData/Local/Programs/Python/Python314:/mnt/c/Users/jhanasri/AppData/Local/Microsoft/WindowsApps:/snap/bin

Search directories:
/home/jhanasri/.local/bin
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/usr/lib/wsl/lib
/mnt/c/Windows/System32
/mnt/c/Windows
/mnt/c/Windows/System32/Wbem
/mnt/c/Windows/System32/Windows PowerShell/v1.0
/mnt/c/Program Files/Git/cmd
/mnt/c/Program Files/Nodejs
/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javase/bin
/mnt/c/Program Files/Java/jdk-21/bin
/mnt/c/Program Files/Microsoft VS Code/bin
/mnt/c/Users/jhanasri/AppData/Local/Programs/Python/Python314/Scripts
/mnt/c/Users/jhanasri/AppData/Local/Programs/Python/Python314
/mnt/c/Users/jhanasri/AppData/Local/Microsoft/WindowsApps
/snap/bin
===== COMMAND RESOLUTION =====
Command 'ls' found at: /usr/bin/ls
Executable permission: YES
Testing existing command: /bin/echo
Command 'echo' found in PATH.
Testing non-existing command: abcxyz
Command 'abcxyz' not found in PATH.
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$
```

===== TEST COMPLETED =====

```
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ which ls
/usr/bin/ls
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ ls -l "$(which ls)"
-rwxr-xr-x 1 root root 142312 Jan 23  2026 /usr/bin/ls
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ command -v ls
alias ls='ls --color=auto'
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ command -v pwd
pwd
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ command -v gcc
/usr/bin/gcc
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ command -v abcxyz
jhanasri@LAPTOP-1RN1M8E3:~/path_practical$ echo $?
```

1

Result:

The PATH variable and command resolution program was successfully implemented and executed in Ubuntu. Commands such as `ls`, `pwd`, and `gcc` were resolved successfully, while the

Observation:

The program correctly searched the directories listed in the PATH variable and identified executable commands. The `which` and `command -v` commands displayed the correct command paths, and the exit status confirmed the failure of the invalid command.